

Python Directed Graph Project Documentation

1. Overview

This project implements a directed graph with integer edge costs and a console-based user interface.

The application supports loading/saving graphs from files, graph editing operations, random graph generation, querying graph properties, and making/restoring a backup copy.

2. Project Structure

- src/main.py

Program entry point. Loads a graph from graph.txt and starts the console menu.

- src/domain/DirectedGraph.py

Core graph model.

Stores:

- `_outbound`: dictionary vertex -> set of outbound neighbors
- `_inbound`: dictionary vertex -> set of inbound neighbors
- `_costs`: dictionary (u, v) -> edge cost
- `_vertices`, `_edges` counters

- src/service/GraphIO.py

File input/output utilities.

- `readGraph(fileName)`: reads graph from text file
- `writeGraph(fileName, graph)`: writes graph in the same format

- src/service/randomGraphMaker.py

Random graph generator.

- `makeRandomGraph(vertices, edges, minCost, maxCost)`

- src/UI/menu.py

Console menu that exposes graph operations to the user.

3. DirectedGraph API

- `isEdge(u, v)`: bool

Checks if edge u -> v exists.

- `addEdge(u, v, cost)`: None

Adds edge if missing; raises `ValueError` if already present.

- `removeEdge(u, v)`: None

Removes existing edge; raises `ValueError` if edge is missing.

- `addVertex()`: int

Adds a new vertex and returns its identifier.

- `removeVertex(vertex)`: None

Removes vertex and all incident edges.

- `setEdgeCost(u, v, cost)`: None

Updates edge cost for an existing edge.

- `getEdgeCost(u, v): int`
Returns edge cost; requires edge to exist.
- `copyGraph(): DirectedGraph`
Deep-copies graph internals (neighbor sets and costs map).
- `parseVertices(): list[int]`
Returns sorted list of existing vertex ids.
- `parseOutboundNeighbors(vertex): list[int]`
Returns sorted outbound neighbors.
- `parseInboundNeighbors(vertex): list[int]`
Returns sorted inbound neighbors.
- `getOutDegree(vertex), getInDegree(vertex): int`
Degree queries for a vertex.
- `getVertices(), getEdges(), getCosts()`
Access graph counters and edge-cost map.

4. File Format

Graph files are text files with this format:

- First line: "<number_of_vertices> <number_of_edges>"
- Next E lines: "<start_vertex> <end_vertex> <cost>"

Example:

```
4 5
0 1 10
0 2 4
1 3 7
2 3 1
3 0 9
```

5. Console Menu Operations

The menu supports:

1. Load graph from file
2. Save graph to file
3. Generate random graph
4. Show graph summary
5. List all vertices
6. Inspect a vertex (degrees + neighbors)
7. Check whether an edge exists
8. View an edge cost
9. Update an edge cost
10. Add a vertex
11. Remove a vertex
12. Add an edge

13. Remove an edge
14. Copy current graph to backup
15. Restore graph from backup copy
16. Print all edges
0. Exit

6. Validation and Error Handling

- Invalid vertex ids produce ValueError.
- Invalid edge access (missing edge) produces ValueError.
- Adding an existing edge produces ValueError.
- readGraph rejects empty files and truncated edge lists.
- Random generator validates negative counts, invalid cost ranges, and edge counts above vertices * vertices.

7. Running the Application

From pythonProject directory:

```
python -m src.main
```

By default, src/main.py tries to load graph.txt and then opens the interactive menu.

8. Notes

- Self-loops are allowed because random generation and edge insertion accept $u == v$.
- Vertices are identified by integers.
- Vertex IDs are not reindexed after deletion; addVertex uses $\max(\text{existing}) + 1$.